

SARAS TECH

Enterprise Product & Growth Audit

Product roadmap, monetization strategy and enterprise-readiness reference

Audit date	2026-09-26
Branch reviewed	feat/homepage-color-type-spacing (on top of 62608f9)
Type	Product / strategy audit. Nothing in this document has been implemented.
Source	docs/ENTERPRISE_PRODUCT_AND_GROWTH_AUDIT.md (this PDF is generated from it)

All prices, costs and competitor statements are hypotheses to validate. The document contains no legal advice; DPDP and content-licensing points require legal review.

Contents

0. Method, confidence, and caveats
1. Executive summary
2. Current product inventory (what already exists)
3. Structural gaps and product limitations (the honest list)
4. Personas and their real pain points
5. Prioritization framework
6. P0 — Foundations required to sell and scale
7. P1 — Features that make institutions and teachers pay
8. P2 — Growth, retention, and enterprise expansion
9. P3 — Long-term / moat
10. Subscription and monetization strategy
11. Indian-market specifics
12. Competitive landscape and differentiation
13. Roadmap
14. Architecture blockers as we scale (detail)
15. Security, privacy, scalability, reliability, billing, and multi-tenant requirements
16. Risks and open questions
17. Appendix: evidence map (where the claims above come from)

Diagrams

- Figure 1 - SarasTech product ecosystem
- Figure 2 - Teacher, Student, Parent, School/Admin workflow
- Figure 3 - Free to Pro to School to Enterprise journey
- Figure 4 - P0 to P3 roadmap
- Figure 5 - Recommended enterprise architecture and multi-tenant flow

How to use this document. It is a long-term roadmap and reference, not a spec. Every feature has a priority (P0–P3), a target user, a payment rationale, and its technical dependencies, so it can be split into tickets. Re-verify the "Current state" claims against the code before acting on them; this repo moves fast (see §0.3).

0. Method, confidence, and caveats

0.1 What was reviewed

- `README.md` , `CLAUDE.md` , `IMPROVEMENTS.md` , and the feature/architecture docs in `docs/` (classroom, classroom mode, generator v2, PYQ plan, learning representation, multimodal attachments, notifications, teacher attendance, mobile plan, Postgres plan, the 2026-07-25 engineering audit, and others).
- `server/prisma/schema.prisma` in full (22 migrations), all route files (~6.5k lines), and the `lib/` , `assistant/` , `actions/` , `safety/` , `learningRepresentation/` modules by structure and targeted reading (flags, budget, limiters, admin analytics, lesson-plan schema, Gemini key pool).
- Client (`client/src/pages` , `components` , `config.ts`) and mobile (`mobile/src/screens`) by directory structure and targeted reading.
- Direct greps for billing/subscription code, a student role, i18n, account deletion, Redis, and the PYQ implementation.

0.2 What was NOT verified

- **No live run.** The app was not started; nothing was exercised in a browser or on a device. Several docs themselves say live Gemini verification is outstanding (Classroom Mode, Generator v2 flags).
- **Not every file was read line-by-line.** Large files (`resources.js` , `teacherAttendance.js` , `index.js` , `index.css`) were read selectively.
- **Competitor claims and pricing** in §12 come from general market knowledge, not from current research. Treat them as hypotheses to validate with a proper competitive teardown.
- **All price points, conversion rates, and cost figures are hypotheses.** Nothing here is a financial forecast. Real Gemini cost per generation has never been measured in this repo (Classroom Mode's doc says cost measurement "needs live data").

0.3 The README is out of date

`README.md` still describes a pilot with 7 models and 4 migrations. The schema now has ~22 models and 22 migrations, and README's feature list omits Classroom management (students, attendance, fees), Teacher Attendance (geofenced, principal-reviewed), Classroom Mode, the AI Action Router, Learning Representations, multimodal attachments, notifications/push, Help & Support, and the mobile app. **This audit relies on the schema, routes, and feature docs, not the README.** Fixing the README is a small P0-adjacent hygiene task because investors, schools, and new engineers will read it first.

1. Executive summary

1.1 Where SarasTech actually is

SarasTech is a **strong teacher-side AI toolkit with the first pieces of a school-operations suite**, built with unusually good engineering discipline (Zod everywhere, owner-scoped 404s, prompt-injection-aware Gemini use, an eval harness, feature flags with kill switches, ~250 mobile tests and a large server suite). It is **not yet an enterprise product**, for four structural reasons:

1. **There is no payment or entitlement layer at all.** No plans, no subscriptions, no usage metering that survives a restart, no invoices. Nothing can be sold today.
2. **Students do not exist as users.** They are name+roll-number rows inside a single teacher's private class. So the product cannot deliver *learning outcomes*, only teacher productivity, and it has no student, parent, or outcome data to build a moat from.
3. **Everything valuable is teacher-private by design.** Classes, students, attendance, fees, and saved resources are scoped to `teacherId` only. There is no school-level roster, no shared library, no co-teaching, no hand-over when a teacher leaves. Schools buy *systems of record*, not personal notebooks.
4. **The scale architecture is still pilot-grade:** SQLite, in-memory rate limits and AI budgets (documented as such), single-process Socket.IO, a pool of several Gemini keys instead of a paid, contractually-backed model tier.

1.2 The single most important strategic decision

Who is the paying customer? The README targets *government-school teachers*, who will not pay individually and buy only through slow B2G procurement. Yet the features built most recently, **student fee tracking, geofenced teacher attendance with principal review, class management, WhatsApp support, exam letterheads**, are *private-school* features. These are the buyers with budget and urgency.

Recommendation: make **low- and mid-fee private K-12 schools (roughly 100–1,500 students, CBSE / ICSE / state board, English- and Hindi/regional-medium)** the primary paying customer (ICP-1), with **individual teachers/tutors** as the bottom-up acquisition funnel (freemium → Pro), and treat **government/NGO/CSR programs** as a separate, grant-funded, low-ARPU channel that validates impact and generates the case studies (ICP-2). This is a recommendation to be confirmed by customer discovery (see §14), not a fact.

1.3 Top recommendations (one line each)

#	Recommendation	Priority
1	Build the billing/entitlement/usage-metering layer so plans and limits exist as data, not env vars	P0
2	Fix the scale/reliability floor: Postgres (when you initiate it), Redis-backed limits/budgets, paid Gemini tier with monitoring, DPDP-grade deletion/export	P0
3	Move classroom data from teacher-private to a school-level roster (academic year, class-teacher-subject assignment, CSV import)	P0
4	Curriculum-grounded generation: board / class / chapter / learning-outcome taxonomy (NCERT + state boards) instead of six coarse grade bands	P1
5	Assessment → marks → report card → remedial loop (the largest recurring teacher time sink that is still unbuilt)	P1
6	Parent communication over WhatsApp (attendance, fees, results) in the parent's language	P1
7	Lightweight student identity (class-code join, assigned practice, auto-graded quizzes, item analysis)	P1

#	Recommendation	Priority
8	School-admin control plane: shared library, HOD review/approval of plans, syllabus-completion tracking, timetable and substitutions	P1 → P2
9	Photo-based grading assist (MCQ first, handwriting later) and PYQ-backed paper builder	P1 → P2
10	Government/enterprise readiness: SSO, audit logs, data residency, APIs, UDISE+/DIKSHA/APAAR integration	P2 → P3

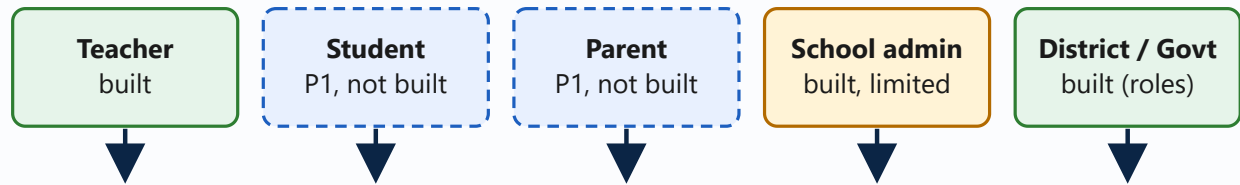
1.4 What NOT to build (yet)

- A generic "AI tutor chatbot for students." Commodity, hard to make safe, no data moat, and the product has no student identity or curriculum grounding to make it better than free options.
- AI image/video generation. The team already (correctly) abandoned "image generation" for structured Learning Representations; keep that discipline.
- Gamification, leaderboards, or badges before there is student data to make them mean anything.
- A marketplace or social network of teachers. Premature until there is a shared library and density.
- Migrating to Postgres *unasked*. `CLAUDE.md` and project memory say this is planned but not sanctioned. This audit recommends it as a P0 dependency, but **the decision and timing remain yours**.

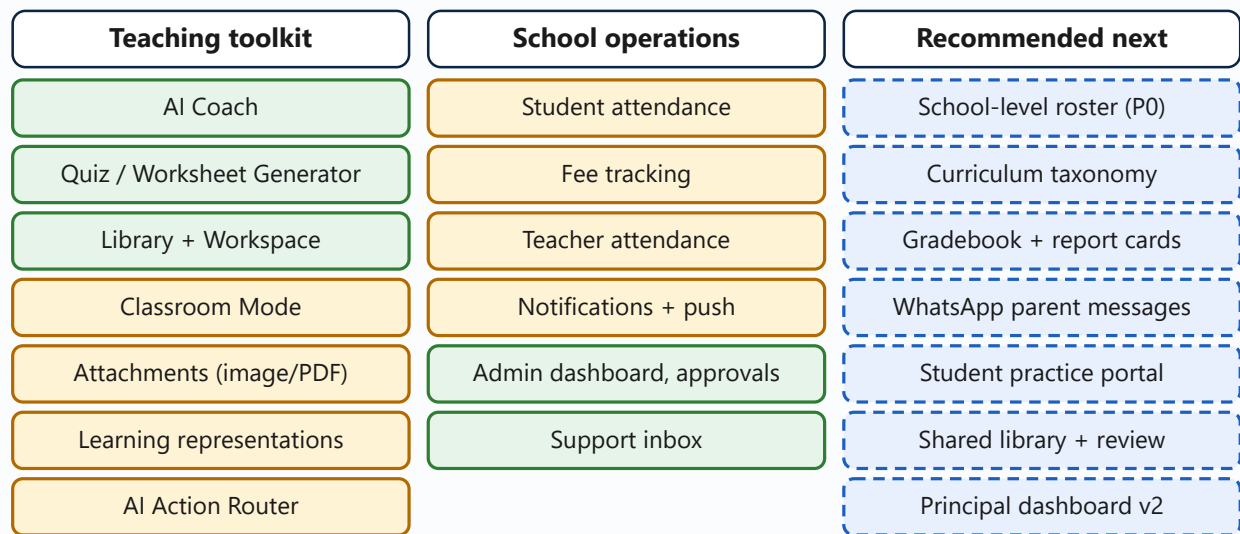
Figure 1 - SarasTech product ecosystem

 Built
 Built, flagged OFF
 Recommended (not built)

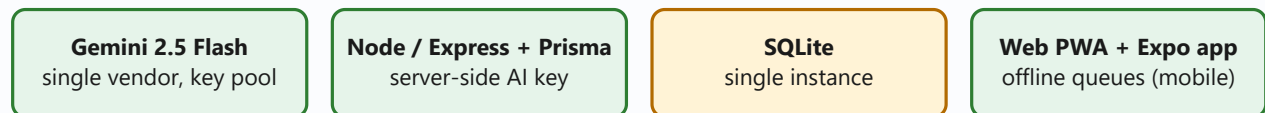
WHO IT SERVES



PRODUCT MODULES



PLATFORM



REQUIRED BEFORE SELLING (P0)



What exists today (green, amber) versus what the audit recommends (dashed blue). Most flagship features are built but switched off.

2. Current product inventory (what already exists)

Status legend: **Built** = code + tests present. **Flagged** = built but default-OFF behind an env flag (needs rollout). **Planned** = design doc only. **Gap** = neither.

2.1 Teacher-facing

Capability	Status	Notes / evidence
AI Coach (Gemini 2.5 Flash), context-aware by grade / subject / classroom type / issue type	Built	POST /api/coach ; server-side prompts, systemInstruction separation

Capability	Status	Notes / evidence
9 Indian languages + Hinglish response language; voice input; read-aloud; WhatsApp share	Built	<code>client/src/config.ts</code> LANGUAGES; browser Web Speech API only
Response-style preference, chat history, pin/rename, feedback 👍 / 🗨️	Built	<code>Query</code> model
Multimodal Coach: up to 5 images/PDFs per message	Flagged (<code>ATTACHMENTS_ENABLED</code>)	Never persisted; magic-byte sniffing
Save to Library, search, type filters (lesson plan / activity / assessment / explanation / general)	Built	<code>Resource</code> model, owner-scoped
Document-style Workspace + AI assist (simplify, add activities, add questions, adapt grade) with preview/apply	Built	<code>POST /resources/:id/ai-action</code>
Quiz/Worksheet Generator with structured Zod-validated questions, deterministic numbering & answer key, LaTeX/KaTeX math	Built (Generator v2 structured mode flagged <code>STRUCTURED_QUESTIONS_ENABLED</code>)	Student/Teacher print variants; fail-closed answer-key omission
Exam-paper letterhead (school, exam, marks, instructions)	Built	<code>ExamHeader*</code>
Classroom Mode: one question → coaching answer + lesson plan / worksheet / quiz / homework / exit ticket	Flagged (<code>CLASSROOM_MODE_ENABLED</code>)	Indian-format lesson plan (objectives, TLM, presentation two-column, blackboard summary). 4 model calls/question. Live Gemini verification outstanding per its own doc
AI Action Router: natural-language command → prefilled generator/navigation	Flagged (<code>ASSISTANT_ENABLED</code>)	Has eval corpus (en/hi/hinglish/nav/adversarial/emergency)
Learning Representations: table, timeline, graph, hierarchy, process, labeled-parts visuals from an answer	Flagged (<code>LEARNING_REPRESENTATION_ENABLED</code>)	Deterministic renderers
Class management: classes, students (name/roll), daily attendance, monthly summary, CSV export	Flagged (<code>CLASSROOM_MANAGEMENT_ENABLED</code>)	Teacher-scoped
Student fee tracking: per-class monthly fee, paid / partial / pending, Excel export	Flagged	<code>FeeRecord</code> ; whole rupees
Teacher attendance: geofenced check-in/out, late/half-day rules, holiday calendar, regularization, principal review, Excel report, reminders	Flagged (<code>TEACHER_ATTENDANCE_ENABLED</code>)	Principal-facing; device id captured
Notifications (in-app, Socket.IO realtime, Expo push on mobile)	Flagged	<code>NOTIFICATIONS_ENABLED</code> , <code>MOBILE_PUSH_ENABLED</code>
Help & Support (bug/feedback, WhatsApp deep link), announcement banner, onboarding	Built	<code>SupportTicket</code> , admin inbox

Capability	Status	Notes / evidence
Native mobile app (Expo/RN) with offline queues for attendance/classroom	Built (push delivery unverified on device; deployment blocked by a hosting-billing lapse per README)	Known bug: offline app restart signs the user out
Installable PWA, dark mode, SEO pages	Built	

Important: most differentiating features are **built but flagged off**. The bottleneck is *validation and rollout*, not more code. See §13 (P0).

2.2 Admin / institution-facing

Capability	Status
Four roles: <code>teacher</code> , <code>school_admin</code> , <code>resource_person</code> (district), <code>super_admin</code>	Built; hard-coded in several places
Role-scoped analytics dashboard (queries by day/subject/issue/language, helpful ratio, top questions)	Built, but aggregates up to 5,000 raw rows in Node (<code>admin.js</code> , <code>take: 5000</code>), so numbers are silently truncated for large tenants
User management, approval queue (every sign-up is <code>pending</code>), role change, session revocation	Built
School creation (super admin), school-code sign-up tenancy	Built
Admin settings / runtime feature flags and AI access by role	Built (global, not per-tenant)
Support inbox with notes	Built
Teacher attendance review + config (principal)	Flagged
Announcements / broadcast notifications	Flagged

2.3 Student-facing

None. There is no student role, login, dashboard, practice, or submission. The only student representation is `Student{name, rollNumber, details}` owned by one teacher.

2.4 Explicitly planned but not built

- **PYQ Question Paper Intelligence** (`docs/pyq-implementation-plan.md`): planning only; no schema, no routes.
- Postgres migration (plan only); git-history secret purge runbook (not executed).
- Classroom Phase 5 reports UI (backend analytics exist).
- SMS/feature-phone, additional languages, video micro-learning (README roadmap).

3. Structural gaps and product limitations (the honest list)

#	Gap	Evidence	Why it matters commercially
G1	No billing, plans, entitlements, or usage ledger	Grep for subscription/billing/payment across <code>server/src</code> , <code>client/src</code> , <code>mobile/src</code> finds nothing relevant	Cannot charge anyone

#	Gap	Evidence	Why it matters commercially
G2	No student/parent identity	Roles are the four in <code>lib/roles.js</code> ; Student has no login	No outcome data, no student value, no parent channel
G3	Teacher-private data model	<code>SchoolClass / Student / AttendanceRecord / FeeRecord</code> filter on <code>teacherId</code> only; <code>schoolId</code> is "reserved... unused by any V1 route"	No principal view of classes/students; teacher leaves = data orphaned; no co-teachers/subject teachers
G4	Resources are owner-only with no sharing	Owner-scoped 404 by design	No school library, no HOD review, no collaboration, no network effect
G5	Coarse curriculum model	<code>GRADES = ['Pre-Primary', 'Class 1-2', 'Class 3-5', 'Class 6-8', 'Class 9-10', 'Class 11-12']</code> , 7 broad subjects, no board, no chapter, no learning outcome	Output is generic, cannot be syllabus-aligned, cannot power assessments/PYQ/remediation
G6	No marks / gradebook / report card	No model	The biggest recurring teacher workload is unaddressed
G7	UI chrome is English-only	No i18n library in client or mobile (the 2026-07 audit's P1 finding still stands); only AI <i>responses</i> are multilingual	Undercuts the core promise for vernacular-first teachers and parents
G8	No account/data deletion or export	No delete route for users; privacy policy promises deletion "on request"	DPDP Act exposure, especially with children's data
G9	Rate limits and AI budgets are per-process memory	<code>assistant/budget.js</code> documents that it resets on deploy and multiplies per instance	Cost-control and abuse-control break on horizontal scale
G10	SQLite + migrate-on-start	<code>server/prisma/schema.prisma</code> , <code>npm start</code>	Single instance; concurrent deploys race; no PITR
G11	Socket.IO without a shared adapter	<code>socketServer.js</code> notes no Redis pub/sub	Realtime breaks across instances
G12	Single AI vendor, multi-key pool	<code>docs/multiple_api_key.md</code>	Quota rotation is not an SLA. A paid product needs a contractual tier, a fallback provider, and a review of the provider's terms on pooling keys
G13	Analytics computed by pulling rows into Node	<code>admin.js</code>	Wrong at scale; no per-school usage reports for the buyer
G14	Feature flags are global + school-code allowlists	<code>lib/flags.js</code> , <code>SystemSetting</code>	Fine for rollout; cannot express "Plan X gets feature Y"
G15	Duplicated vocabularies across server/web/mobile ("CHANGE-11")	Comments in <code>config.ts</code> , <code>roles.js</code>	Every new role/language/board costs three edits; drift risk
G16	Print-only "export"	Browser print dialog	Teachers need real DOCX/PDF/Excel and WhatsApp-ready files
G17	Analytics say "queries," not outcomes	<code>/admin/analytics</code>	A principal cannot see syllabus coverage, learning gaps, teacher workload
G18	Offline session restore bug on mobile	README "Known Issue"	Poor-connectivity users (the target market) get signed out

4. Personas and their real pain points

4.1 Teacher (government / low-fee private, often 1 teacher : 40–70 students, sometimes multi-grade)

Time sinks, in rough order of hours per week: **(1) writing tests/worksheets and checking them; (2) preparing marks registers, term sheets and report cards; (3) lesson-plan paperwork demanded by supervisors (many teachers must submit plans, in a prescribed format); (4) attendance/fee registers; (5) parent follow-up; (6) remediation for weak students with no time.** SarasTech today addresses (1) partly, (3) well, (4) partly, and not (2), (5), (6).

4.2 Student

Needs practice that matches their level, feedback in their language, and exam-pattern familiarity (board exams, Olympiads, entrance tests for older grades). SarasTech serves none of this directly.

4.3 Parent

Wants to know: was my child present, what are the marks, what fee is due, what to do at home. In India the channel is **WhatsApp**, in the parent's language, often on a shared or low-end phone.

4.4 School admin / principal

Wants control and visibility: are teachers present, are plans prepared, is the syllabus on track, how are classes performing, who has unpaid fees, what to tell the management/trustees/inspector. Wants **one system of record**, not thirty private teacher notebooks.

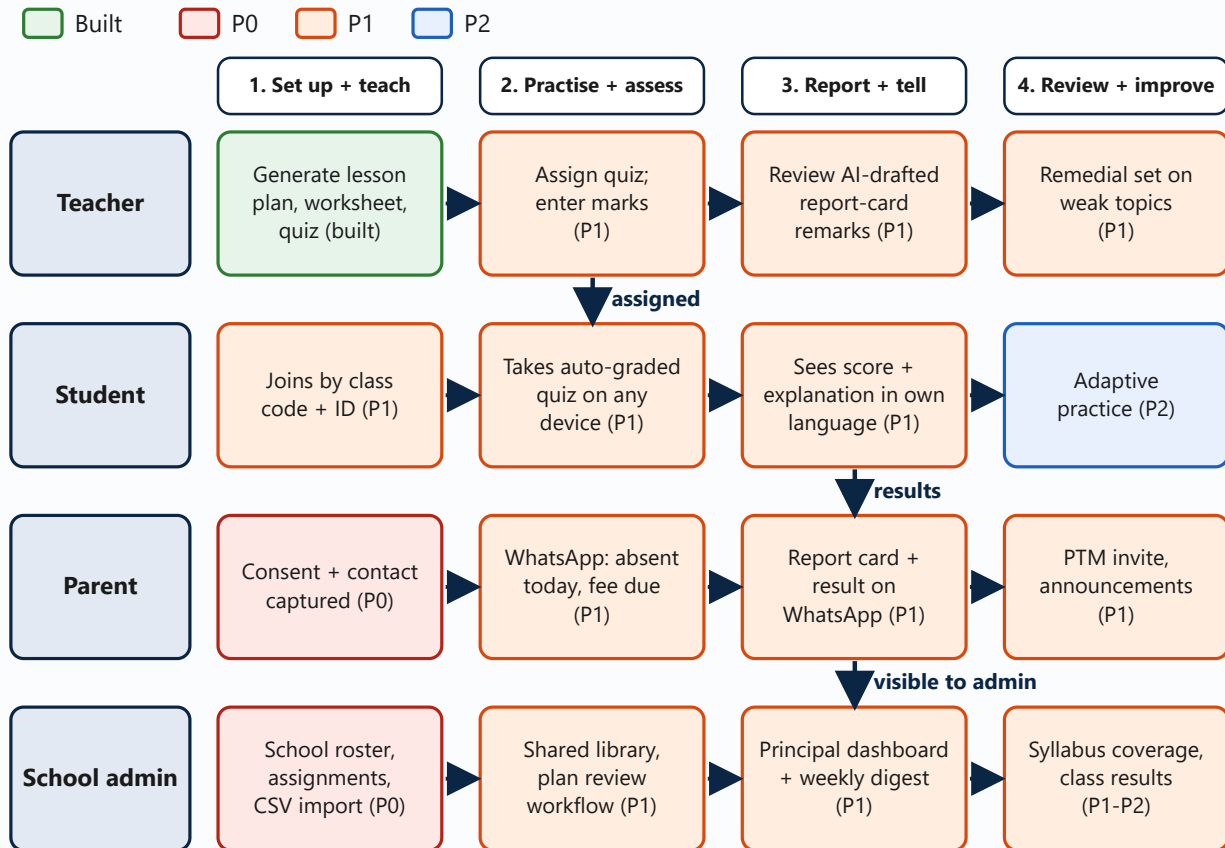
4.5 Paying institution / group / trust

Wants per-school and cross-branch reporting, predictable per-student or per-school pricing, data ownership, audit trails, SSO, and a vendor that will exist next year.

4.6 Government / NGO / CSR buyer

Wants evidence of impact at low cost per teacher, alignment to NEP 2020 / NIPUN Bharat / TaRL / state curricula, Hindi and regional-language usability, offline-tolerant operation, and data hosted in India.

Figure 2 - Teacher, Student, Parent, School/Admin workflow



Vertical arrows show hand-offs between roles. Teacher and admin steps rely on the P0 school-level data model; student and parent steps depend on P0 consent and identity foundations.

Recommended end-to-end workflow. Only teacher content generation is fully built today; the rest are P0/P1/P2 recommendations.

5. Prioritization framework

Tier	Meaning	Gate
P0	Must exist <i>before</i> charging money or onboarding schools at scale	Without it: cannot bill, cannot safely scale, or legal/security exposure
P1	The features that make a school or Pro teacher say yes to paying	Directly saves hours weekly or produces measurable outcomes
P2	Growth and retention multipliers; enterprise expansion	Builds on P1 data
P3	Long-term moat and large-contract features	Needs scale, data, or partnerships

Each recommendation below states: **Problem · Target user · Why they'd pay · Impact · Priority · Dependencies.**

6. P0 — Foundations required to sell and scale

P0-1. Subscription, entitlement, and usage-metering layer

- **Problem:** Nothing in the codebase can express "this school is on the Standard plan with 25 teacher seats and 500 AI generations/teacher/month."
- **Target user:** Internal (finance/ops), school admin (sees plan, usage, invoices), teacher (sees remaining quota).
- **Why they'd pay / why it matters:** It is the enabler of every paid plan. Also protects margin: AI cost is a per-call variable cost, so unmetered AI on a flat price loses money.
- **Impact:** Unlocks revenue; enables free-tier abuse control and honest upsell prompts.
- **What it includes:** `Plan`, `Subscription` (tenant = school, or teacher for individual plans), `Entitlement` set (feature booleans + numeric limits), persistent `UsageLedger` (per user/school/feature/day; AI call counts and token estimates), payment provider integration (Razorpay first for India: UPI, cards, netbanking, and e-mandate/autopay; invoices with GST), trial / grace / dunning states, admin-visible usage page. Evolve `SystemSetting` + `flags.js` into a resolver: *flag on globally AND tenant plan grants it*.
- **Dependencies:** Persistent DB writes at scale (Postgres strongly preferred), Redis for hot counters, GST invoicing/legal entity, decision on tenant model (school vs. teacher-as-tenant for individuals).
- **Note:** Replaces the process-local `budget.js` and per-IP AI limiters as the *business* limit. Rate limiting stays as *abuse* protection.

P0-2. Data platform and reliability floor

- **Problem:** SQLite, in-memory limiters/budgets, single-process Socket.IO, migrate-on-start.
- **Target user:** Everyone; buyers do due diligence.
- **Why it matters:** A school will not pay for a system that loses attendance data or goes down at 9 a.m. on a Monday (the exact peak for attendance and check-in).
- **Actions:** (a) PostgreSQL (managed, with PITR and backups; follow `docs/postgres-migration-plan.md`; **you must initiate this, per the standing project constraint**); (b) Redis for rate limits, AI budgets, and Socket.IO adapter; (c) run migrations as a release step, not on every instance start; (d) health/readiness checks and graceful shutdown; (e) structured logging, error tracking, uptime alerts, an SLO for check-in/attendance endpoints; (f) load test the 8:30–10:00 IST check-in burst and Monday-morning login burst.
- **Priority:** P0. **Dependencies:** managed Postgres and Redis hosting, budget, staging environment. Also resolve the hosting-billing lapse that blocked mobile push verification. That is an operational risk in itself.

P0-3. Production-grade AI supply

- **Problem:** Multiple Gemini keys rotated by quota/time (`geminiKeyPool.js`), all on one vendor, with quota reset assumptions tied to a clock time.
- **Why it matters:** Paid customers need predictable latency and contractual terms; pooling keys to stay under quotas is fragile and may conflict with the provider's terms. Verify this with the provider's current terms before relying on it commercially.
- **Actions:** Move to a paid, billing-backed tier (Gemini API paid / Vertex AI in an India region if data-residency is required); add cost and latency dashboards per feature; add a **secondary provider fallback** behind the existing `gemini.js` seam; cache deterministic outputs where safe; measure real cost per generation for each feature (Classroom Mode = 4 calls/question). Keep the existing safety posture (system instruction vs. untrusted content) unchanged.
- **Priority:** P0. **Impact:** Gross-margin visibility; no rate-limit outages on launch day.

P0-4. Privacy, consent, and data-lifecycle compliance (DPDP Act 2023 direction)

- **Problem:** Children's data (names, roll numbers, attendance, fee status) is stored; there is no user-deletion route, no export, no consent capture, no retention policy, no audit of data access.
- **Why it matters:** Schools' legal/management teams will ask. The DPDP Act requires verifiable parental consent for processing children's data and gives erasure/correction rights. **Get legal advice on exact obligations. This document is not legal advice.**
- **Actions:** self-serve and admin-initiated account deletion with defined cascade/anonymization (the schema already prefers `SET NULL`); school-level data export; consent records (school as data fiduciary vs. SarasTech as processor: a Data Processing Agreement template); retention windows for `Query`, `Event`, attachments (never stored today, good); ensure student names/PII are **never sent to the model** (test for it); PII redaction in support tickets/logs; India data-residency option; breach-response runbook.
- **Priority:** P0. **Dependencies:** legal review; decision on controller/processor roles.

P0-5. School-level data model (from teacher-private to institutional)

- **Problem:** G3/G4 above.
- **Target user:** School admin/principal (buyer), teachers (co-teaching, transfers).
- **Why they'd pay:** This is what turns "a teacher's app" into "our school's system." It is the prerequisite for every admin feature and for reporting the buyer values.
- **What it includes:**
 - `AcademicYear`, school-owned `Class / Section` roster; `Student` as a school-owned entity with admission number, DOB/age, guardian contacts (consent-gated), and a stable ID (later map to APAAR ID).
 - `TeacherAssignment` (teacher × class × subject × year), replacing "class owned by one teacher." Class-teacher vs. subject-teacher permissions.
 - Bulk **CSV/Excel import** of students and teachers, with validation and a dry-run preview (schools will not hand-type 800 students).
 - Year-end rollover/promotion; class transfer; teacher hand-over.
 - Tenant isolation tests generalized from `tenant-isolation.test.js` / `classroom-tenant-isolation.test.js`.
 - Keep the existing 404-not-403 discipline.
- **Dependencies:** P0-2 (Postgres); a careful data migration path for existing teacher-owned classes (map each teacher-owned class to a school-owned class + assignment). This is the riskiest schema change on the roadmap; do it before more features pile onto `teacherId`-scoped tables.

P0-6. Launch-readiness of already-built features

- **Problem:** Most flagship features are flagged off and several were never verified against live Gemini or on a device.
- **Actions:** live-verify Classroom Mode (cost, latency, math notation, four-call batching), Generator v2 structured mode, attachments, push notifications; fix the mobile offline-restore sign-out bug; decide which flags become defaults; refresh the README; add a public "what's included" page.
- **Priority:** P0 because it is cheap, unblocks real pilots, and produces the usage and cost data every pricing decision in \$10 depends on.

P0-7. Quality and trust baseline for AI content

- **Problem:** Teachers will print AI-authored questions and answer keys. A wrong answer key in front of 40 students is a churn event.
 - **Actions:** extend the existing eval harness (`server/evals`) with **answer-key correctness** and **curriculum-accuracy** suites for Mathematics/Science in Hindi and English; automatic self-consistency check on answer keys (solve-then-compare) before showing; a visible "AI-generated: review before use" affordance and one-click "report wrong answer" feeding the eval corpus; a regression gate in CI for prompt changes.
 - **Why they'd pay:** Trust. This quietly becomes a differentiator versus generic chatbots.
-

7. P1 — Features that make institutions and teachers pay

P1-1. Curriculum-grounded generation (board, class, chapter, learning outcome)

- **Problem:** Output is aligned only to a coarse band. A Class 7 CBSE teacher on Chapter 3 gets a generic "Class 6-8 Maths" answer. Teachers, principals, and inspectors evaluate content by **chapter and learning outcome**.
- **Target user:** Teachers (daily), principals (curriculum compliance), parents/students (exam relevance).
- **Why they'd pay:** Trusted, syllabus-aligned material saves rewriting and is the difference between "toy" and "tool."
- **Impact:** High: raises accuracy, enables PYQ, gradebook-by-topic, remediation, syllabus tracking.
- **What it includes:** A curriculum taxonomy (Board → Class → Subject → Chapter → Topic → Learning Outcome/NCERT LO or NIPUN competency) for CBSE/NCERT first, then 3–5 large state boards by user demand; selectable per class/subject; prompts grounded in the chapter's topics; optional retrieval over licensed/open textbook text (**verify NCERT/state content licensing before ingestion**); per-class default board/subject so teachers do not re-select each time.
- **Dependencies:** taxonomy data sourcing and licensing; P0-5 (class-level board/subject); eval corpus for curriculum accuracy.
- **Priority:** P1 (start immediately after P0-5; the data-sourcing work can begin now).

P1-2. Marks entry → gradebook → report card → remediation loop

- **Problem:** Teachers spend large parts of term-end on marks registers and report cards. SarasTech creates assessments but never records the results, so it forgets the most valuable data.
- **Target user:** Teachers (time), principals (consolidated results), parents (report card).
- **Why they'd pay:** It replaces a painful, error-prone manual process every term and is the feature that makes a principal say "we need this for all teachers."
- **What it includes:**
 1. Link an assessment (already a `Resource`) to a class and record marks per student (quick grid entry, mobile-friendly, offline queue: the mobile app already has an offline queue pattern).
 2. Configurable grading schemes (marks/grades, board-style internal + external components, term weights).
 3. **Report-card PDF** with school letterhead (reuse `ExamHeader`), in the parent's language, with **AI-drafted, teacher-editable remarks** (the child's strengths/areas to improve, in Hindi/regional language). Remarks are a well-scoped, high-value, low-risk generative task.
 4. Item analysis: which questions the class missed → one-click "generate remedial worksheet on these sub-topics."

- **Dependencies:** P0-5 (school student roster), P1-1 (topic tags per question), true PDF generation (server-side, not browser print).
- **Priority:** P1: likely the single highest-ROI feature for the school buyer.

P1-3. Parent communication over WhatsApp

- **Problem:** Teachers phone/message parents manually about absence, fees, results, meetings.
- **Target user:** Teachers and admin (time saved), parents (the real end-consumer).
- **Why they'd pay:** Directly reduces fee-collection delay (schools care intensely) and absenteeism; visible to parents, which sells the school.
- **What it includes:** Consent-gated guardian contacts; WhatsApp Business API (per-message costs are real; model them into pricing) with approved templates: absent today, fee due/receipt, exam result, PTM invite, announcement, in the guardian's preferred language; teacher/admin approves batch sends; delivery status; opt-out; SMS fallback where WhatsApp is unavailable. The existing notification service (`notificationService.js`) is the natural dispatch seam.
- **Dependencies:** WhatsApp Business provider (BSP) account and template approvals; consent (P0-4); P0-5 contacts; per-message cost metering (P0-1).
- **Priority:** P1.

P1-4. Minimal student identity and assigned practice

- **Problem:** No student-side value or data (G2).
- **Target user:** Students (practice), teachers (auto-graded homework and item analysis without checking notebooks), parents.
- **Why they'd pay:** Auto-graded assignments save the checking time teachers hate most; class-level analytics create outcome evidence for the school and for renewals.
- **What it includes (deliberately small):**
 - Class-code join with a teacher-approved roster match, **no email required** (many children have none): school-issued student ID + PIN or parent-phone OTP; a `student` role with the narrowest possible scope (own assignments/results).
 - Teacher assigns a generated quiz (MCQ/true-false auto-graded; short answers flagged for teacher review) with a due date.
 - Student takes it on a phone (mobile + PWA, low-bandwidth, works offline and syncs); sees score and explanation **in their language**.
 - Teacher sees per-student and per-question results (feeds P1-2).
- **Risks:** child-safety and consent (P0-4); shared-device sessions; do **not** start with open-ended chat for students.
- **Dependencies:** P0-1, P0-4, P0-5, P1-1; new role plumbing (touches the duplicated role lists, G15, so consolidate those into a shared package first).
- **Priority:** P1 (MVP scope only); the rich student experience is P2.

P1-5. Shared school library and lesson-plan review workflow

- **Problem:** Resources are owner-only (G4); supervisors demand plan submission and review, which today is done on paper/WhatsApp.
- **Target user:** Principal/HOD/coordinator (review), teachers (reuse, less rework).
- **Why they'd pay:** Institutional memory: a departing teacher's plans stay; new teachers start from the best plans. Review/approval workflow replaces paper submissions. This is a classic seat-expansion feature.

- **What it includes:** Share a resource to a class/department/school with view/copy permissions; versioned copies; plan status (draft → submitted → approved / changes requested) with comments; HOD/coordinator role; school-wide template for the lesson-plan format the school prescribes (the Indian government format is already implemented; private schools often have their own).
- **Dependencies:** P0-5 roles/assignments; permissions model beyond four hard-coded roles.
- **Priority:** P1.

P1-6. Real exports (DOCX, PDF, Excel) and WhatsApp-ready sharing

- **Problem:** Print-to-PDF via the browser is the only output. Teachers edit in Word, share PDFs on WhatsApp, and schools archive files.
- **Why they'd pay:** Removes a daily friction; "works with my existing workflow" is a top adoption factor.
- **What it includes:** Server-rendered PDF and DOCX for worksheets/quizzes/lesson plans/report cards with letterhead and Devanagari/regional fonts (font handling is the hard part); student and teacher versions preserved (the fail-closed answer-key logic must carry over); Excel exports already exist for attendance/fees; optional shareable read-only link.
- **Dependencies:** headless rendering service or a document library; font licensing; storage for generated files if links are offered.
- **Priority:** P1.

P1-7. Principal dashboard v2 (from "AI usage" to "school health")

- **Problem:** Current analytics count queries. A principal needs teacher attendance summary, plan-submission status, assessments conducted, class-wise results, syllabus coverage, fee collection, and low-adoption teachers.
- **Why they'd pay:** It is the buyer's daily-open screen and the evidence for renewal.
- **What it includes:** Server-side aggregation (fix G13 by pre-aggregated tables or SQL group-bys rather than 5,000-row JS reduction); scheduled weekly e-mail/WhatsApp digest to the principal; drill-down by class/teacher; export for trustees/inspection.
- **Dependencies:** P0-2, P0-5, P1-2 data. Some panels (teacher attendance, fee totals) can ship earlier from existing data.
- **Priority:** P1, with a P0-adjacent quick fix for the truncated analytics.

P1-8. UI localization (Hindi first, then 3–4 regional)

- **Problem:** Only AI answers are multilingual; menus and buttons are English (G7).
- **Target user:** Vernacular-medium teachers, parents, students.
- **Why they'd pay / adopt:** It is often a hard adoption blocker in Hindi-belt, Tamil Nadu, Karnataka, Maharashtra, and Bengal schools.
- **What it includes:** i18n library on web and mobile with a shared key catalog; Hindi first; then by customer demand; keep AI-content language (per-resource) independent of UI language; native-speaker review, not raw machine translation.
- **Priority:** P1 (Hindi), P2 (others).

8. P2 — Growth, retention, and enterprise expansion

P2-1. PYQ-backed paper builder and blueprint-driven exam papers

- **Problem:** Board-class teachers want papers resembling real exams, with marks distribution and question-type weightage (blueprint).
- **Status:** Already designed in `docs/pyq-implementation-plan.md` (deterministic selection, human review queue, provenance). Not built.
- **Why they'd pay:** Board-exam preparation is the highest-willingness-to-pay segment (Class 9–12, coaching/tuition).
- **Recommendation:** Build the **blueprint paper builder** (marks split by chapter/Bloom level/difficulty, section-wise, total marks constraint) **before** PYQ ingestion; it needs no copyrighted corpus and depends only on P1-1. Treat PYQ ingestion as a content-operations investment (human review of extracted questions is a *cost center*: budget for it) and **verify copyright/licensing of past papers** before ingesting.
- **Dependencies:** P1-1; content ops team; legal.

P2-2. Photo-based grading assist

- **Problem:** Checking 40 answer sheets nightly.
- **Approach, in order of risk:** (1) **MCQ/OMR-style sheets scanned via phone camera** (deterministic, high accuracy; realistic soon); (2) short numeric/one-word answers; (3) handwritten descriptive answers with AI-suggested marks against a rubric, **always teacher-confirmed**, in Hindi/English. The multimodal attachment pipeline (`gemini.js` multimodal, `fileValidation.js`) is a reusable foundation.
- **Why they'd pay:** Hours saved per week per teacher; high perceived value; strong differentiation if accuracy is real.
- **Risks:** handwriting/regional-script accuracy; teacher trust; per-image AI cost (needs metering, P0-1). Ship with an accuracy dashboard and per-school opt-in.
- **Priority:** P2 (MCQ scan may pull forward to late P1 if pilots demand it).

P2-3. Adaptive practice and personalized remediation for students

- **Problem:** Single-pace instruction; weak students fall behind (TaRL/NIPUN principles already cited in the README).
- **What it includes:** Per-student topic mastery (from P1-2/P1-4 data); auto-generated remedial sets at the child's level; teacher-facing grouping suggestions ("these 8 students need fractions revision"); spaced re-testing; FLN baseline/endline assessments for primary grades aligned to NIPUN Bharat competencies.
- **Why they'd pay:** Measurable improvement claims; NGOs/state programs pay for FLN outcomes.
- **Dependencies:** P1-1, P1-2, P1-4 data volume; careful pedagogy review.

P2-4. Timetable, substitution, and syllabus-coverage tracking

- **Problem:** Principals manually assign substitutes each morning; no visibility on syllabus pace.
- **What it includes:** Period-wise timetable; absence (from teacher attendance, already built) → suggested substitutes → notification to the substitute with an AI-generated ready-made 40-minute cover lesson from the absent teacher's plan or the topic; teacher marks chapter/topic taught → coverage dashboard vs. annual plan.
- **Why they'd pay:** Uniquely leverages the existing **teacher-attendance + lesson-plan** assets; a compelling, hard-to-copy bundle for principals.

- **Dependencies:** P0-5, P1-1, teacher attendance in production.

P2-5. Student-facing AI tutor (guarded), in the student's language

- **Problem:** Doubts outside school hours; no tutor access for most children.
- **Constraints:** Curriculum-grounded (P1-1), teacher-visible transcripts, per-school opt-in, strict safety filters (the input/output guards exist), daily quotas (P0-1), read-aloud in the student's language. Ship only after student identity and consent are solid.
- **Why they'd pay:** Parents/students pay for "doubt solving"; schools pay for a *safe, supervised* version.
- **Priority:** P2.

P2-6. Enterprise identity and administration

- **Includes:** SSO (Google Workspace / Microsoft; SAML/OIDC for larger groups), SCIM or bulk provisioning, configurable roles/permissions beyond the four hard-coded ones, immutable audit log (attendance already has an activity log to emulate), IP allow-lists optional, admin impersonation with audit (support), data-export APIs, webhooks.
- **Why:** Multi-branch chains and trusts require it; procurement checklists demand it.

P2-7. Multi-branch and group-of-schools management

- **Problem:** Trusts/chains with 5–50 schools want cross-branch dashboards, common templates, and centrally managed billing.
- **What it includes:** Organization → Schools hierarchy (today `School` is the top tenant; add an `Organization` above it), central plan/billing, cross-branch analytics, shared template library, org admin role. The `resource_person` (district) role is a precedent for cross-school scoping.
- **Priority:** P2 (build when the first 2–3 chain prospects appear).

P2-8. Teacher professional development (CPD) micro-learning

- **Problem:** NEP 2020 expects ~50 hours of CPD per teacher per year; the Coach already gives just-in-time advice but is not tracked.
- **What it includes:** Short, voice-friendly micro-courses seeded from real classroom problems (the Coach "issue types" and analytics show the top pain points), completion certificates (verifiable, for school/district records), district dashboards for resource persons.
- **Why they'd pay:** Government/NGO buyers and trusts fund CPD; certificates motivate teachers.
- **Priority:** P2 (also the natural B2G/CSR story).

P2-9. Better voice and audio for low-literacy and low-bandwidth use

- **Problem:** Browser Web Speech API quality varies widely and is weak or absent for several Indian languages on many low-end Android devices.
- **What it includes:** Server-side or provider-backed speech-to-text and text-to-speech for the 9 supported languages; short audio summaries of lesson plans; WhatsApp voice-note replies.
- **Dependencies:** provider evaluation for language coverage; cost metering.

9. P3 — Long-term / moat

Feature	Problem	Why long-term
Government integrations (UDISE+ codes, DIKSHA content/QR mapping, APAAR ID, state MIS export formats)	Duplicate data entry into state systems	Needed for B2G scale; depends on API access and approvals
B2G state/district deployments (data residency in India, on-prem/state-cloud option, tender-grade security docs, VAPT reports)	Government procurement requirements	Large but slow contracts; only after P0/P1 are solid
Teacher-created content marketplace	Monetize best content; network effects	Needs shared library density (P1-5) first
Predictive early-warning (attendance + marks → dropout/at-risk)	Prevent dropouts	Needs 1–2 years of data and careful ethics review
Alumni/board-result analytics for institutions	Marketing outcomes	Depends on longitudinal data
Open API / partner ecosystem (LMS, ERP, fee gateways)	Fit into school tech stacks	Once core stabilizes
White-label for large chains/publishers	Additional revenue	Needs multi-tenant theming and support model
Content partnerships (publishers, NGOs) for licensed question banks and textbooks	Content is the moat	Legal/BD, not engineering

10. Subscription and monetization strategy

All prices are placeholders to test, not recommendations to publish. Set them from customer discovery, the measured cost per AI generation, and WhatsApp/SMS costs.

10.1 Principles

1. **Free tier must prove value, not give away the margin.** Free = daily habit + hooks; paid = the time-savers and the school-level system.
2. **Charge on what maps to value the buyer feels:** teacher seats (Pro), students or school size (institution), and metered messaging/AI heavy features (fair-use pool + top-ups).
3. **Sell the system of record to schools; sell productivity to teachers.** Two motions, one product.
4. **Never gate safety, privacy, or basic access to one's own data behind payment.**

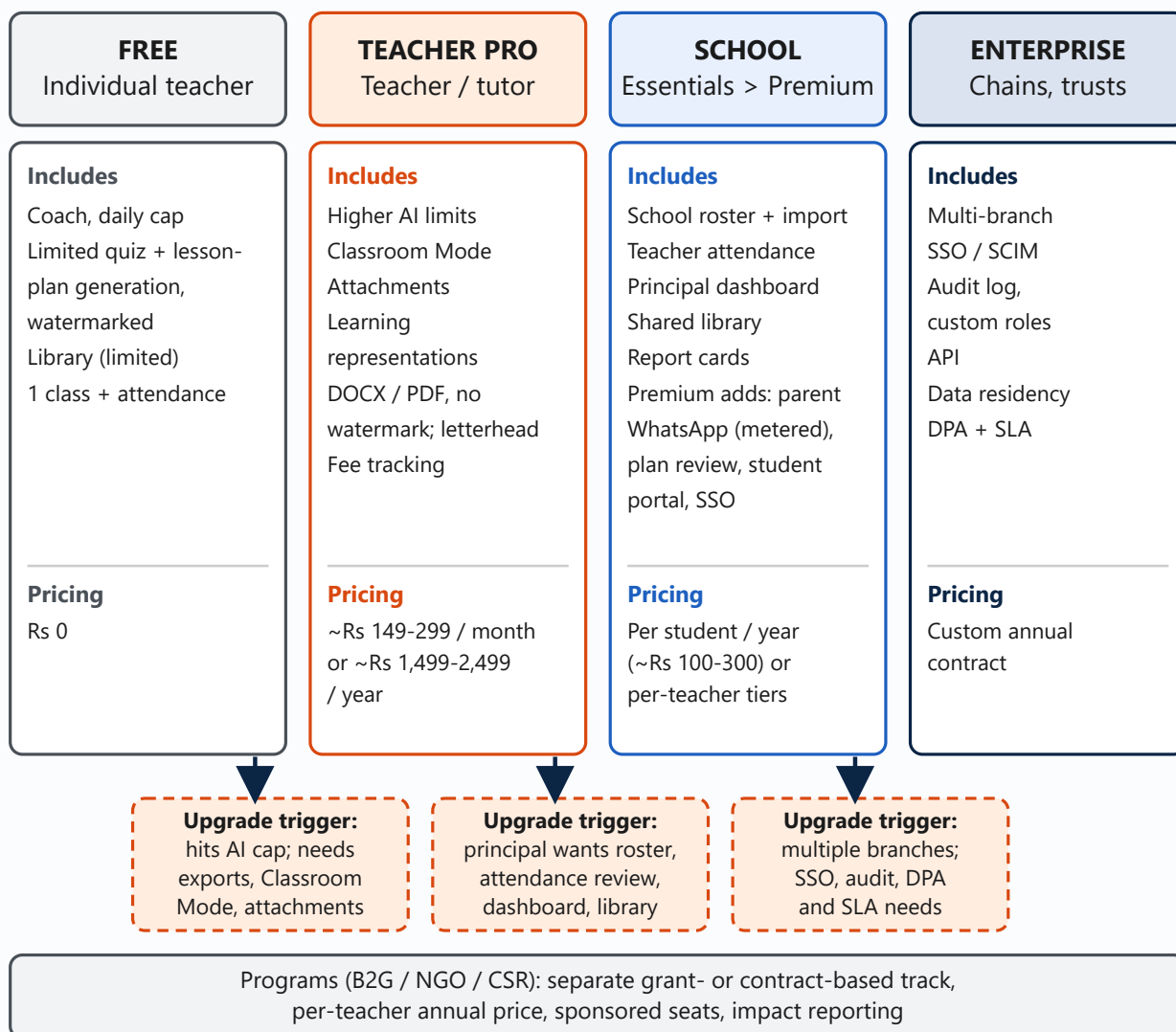
10.2 Proposed plans (hypotheses)

Plan	Buyer	Includes (summary)	Pricing hypothesis (to test)
Free (Teacher)	Individual teacher	Coach with daily cap; basic lesson-plan and quiz generation (watermarked / limited count); Library (limited items); 1 class with attendance; community/Help	₹0

Plan	Buyer	Includes (summary)	Pricing hypothesis (to test)
Teacher Pro	Individual teacher / tutor paying out of pocket	Higher/unlimited-fair-use AI; Classroom Mode; multimodal attachments; DOCX/PDF export without watermark; letterhead; full Library; multi-language; learning representations; report-card remarks up to N students	~₹149–₹299/month or ~₹1,499–₹2,499/year (annual is more realistic for India; UPI autopay)
School Essentials	Small/mid private school	All Pro features per teacher seat + school roster, class/attendance/fee system, teacher attendance, principal dashboard, shared library, bulk import, announcements, report cards, standard support	Per-student/year (e.g. ₹100–₹300) or per-teacher/month tiers; minimum annual commitment
School Premium	Larger / quality-focused school	Essentials + parent WhatsApp module (metered), plan review workflow, timetable/substitutions, student practice portal, advanced analytics, priority support, SSO	Higher per-student rate + messaging pass-through
Institution / Group (Enterprise)	Chains, trusts, colleges	Multi-branch, SSO/SCIM, audit logs, custom roles, data-residency, API, DPA/SLA, custom onboarding and training	Custom annual contract
Programs (B2G/NGO/CSR)	State/district/NGO	Per-teacher annual price, sponsored/free seats, impact reporting, local-language content, CPD tracking	Grant/contract-based; low per-teacher price

Figure 3 - Free to Pro to School to Enterprise journey

☐ All prices are hypotheses to test, not final



Proposed plan ladder and upgrade triggers. Pricing is a set of hypotheses to validate through customer discovery.

10.3 Free vs. paid: where to draw the line

Capability	Free	Pro	School plans
Coach Q&A (daily cap)	✓ limited	✓ high	✓ high
Lesson plan / quiz / worksheet generation	Limited count, watermark	✓	✓
Classroom Mode (4 calls per question — expensive)	✗ (or 3/month trial)	✓	✓
Multimodal attachments (image/PDF)	✗ / trial	✓	✓
Learning Representations	✗ / trial	✓	✓
Exports (DOCX/PDF, no watermark), letterhead	✗	✓	✓
Personal class + student attendance	1 class	✓	✓ (school-wide)
Fee tracking	✗	✓	✓
Marks/gradebook, report cards	Basic (view)	✓ limited	✓ full

Capability	Free	Pro	School plans
School roster, bulk import, year rollover	✗	✗	✓
Teacher attendance (geofence, review)	✗	✗	✓
Principal dashboard, digests	✗	✗	✓
Shared library, plan review workflow	✗	✗	✓
Parent WhatsApp/SMS	✗	✗	✓ metered
Student portal / assigned practice	✗	✗	✓
SSO, audit log, custom roles, API, DPA	✗	✗	Enterprise

The AI-cost-heavy features (Classroom Mode's four calls, attachments, learning representations, photo grading) must be gated by **entitlement + metered quota**, not just a flag.

10.4 Unit economics to measure before pricing

- Real cost per generation by feature (Gemini tokens, images/PDF, retries and continuations up to `LLM_MAX_CALLS_PER_REQUEST=8`).
- WhatsApp Business per-message pricing by category; SMS cost.
- Storage/bandwidth for exports (if files are persisted).
- Support cost per school (onboarding and training are often the largest cost in Indian school software).
- Payment fees and GST handling; refunds/chargebacks.

10.5 Go-to-market notes (product-relevant)

- Bottom-up:** free teacher plan with shareable outputs (letterhead + subtle SarasTech attribution on free exports) → teacher asks principal to buy.
- Top-down:** principal pilot of one term in 1–3 schools with weekly digest as the hero deliverable.
- Channels:** teacher WhatsApp communities, school-management associations, state teacher-training partnerships, NGO/CSR programs, education-fair demos.
- Onboarding is the product:** bulk import + a guided 30-minute setup are as important as any feature.

11. Indian-market specifics

Area	Requirement / opportunity	Status
Languages	9 languages + Hinglish for AI content today; add Malayalam, Punjabi, Assamese, Urdu, Kannada quality passes, and Sanskrit/regional-script fonts in print	Partial
UI localization	Hindi first; then regional (P1-8)	Gap
Boards	CBSE, ICSE/ISC, state boards (UP, Bihar, Maharashtra, Tamil Nadu, Karnataka, West Bengal, Rajasthan, Gujarat, MP, AP/Telangana...) with board-specific patterns and terms (FA/SA, CCE-style internal assessments, term structures)	Gap
Medium of instruction	Textbooks in Hindi/regional medium; generated material must match the <i>textbook's</i> terminology, not machine-translated	Gap
Multi-grade classrooms	Already a "classroom type" option in the Coach (a real differentiator for government/rural and small schools); extend to multi-grade lesson plans and worksheets with tiered difficulty in one document	Partial

Area	Requirement / opportunity	Status
Low connectivity / low-end phones	PWA + offline queues exist for attendance; extend offline to marks entry and student quizzes; keep bundle small (code-splitting was a flagged issue); test on ₹6–8k Android devices	Partial
Shared devices / no email for children	Student ID + PIN/parent-OTP; no email requirement	Gap
Payments	UPI, autopay (e-mandate), annual plans, GST invoices, purchase orders for institutions	Gap
Fee workflows	Monthly fee tracking exists; add receipts, dues reminders, concessions/siblings, fee heads (tuition/transport/exam), payment links (UPI)	Partial (basic paid/partial/pending)
Government schemes/formats	Teacher lesson plan format (built); mid-day meal counts, UDISE+ data, DIKSHA QR alignment, NIPUN Bharat FLN assessments, TaRL grouping	Mostly gap
Exams	Board-pattern blueprints, PYQ, Olympiad/NTSE/scholarship prep, competitive-exam practice for Class 9–12	Gap (PYQ planned)
Festivals/calendar	Holiday lists by state; academic calendar; existing holiday model can be extended	Partial
Trust/branding	School letterhead, principal signature block, school logo on all outputs	Partial (text letterhead only)

12. Competitive landscape and differentiation

Caveat: Based on general market knowledge; validate each row with a current teardown.

Category	Examples (illustrative)	Their strength	Their gap SarasTech can exploit
Generic AI chat	ChatGPT, Gemini, Claude apps	Cheap/free, powerful	Not curriculum-aligned, no classroom/school context, no Indian formats, no print-ready or role-based workflows
Global teacher-AI tools	MagicSchool-type products	Polished teacher tools	English/Western-centric, not Indian boards/languages/formats, USD pricing, no fee/attendance/school ops
Indian school ERP/management	Teachmint, Classplus, Entab, Fedena-type products	Fees, attendance, communication, admissions	AI content generation and pedagogy are weak or bolted-on
Indian learning/test-prep	PhysicsWallah, Vedantu, Embibe-type	Strong student-facing content and brand	Student-first (B2C), not a teacher-workflow or school system-of-record product
Government platforms	DIKSHA and state portals	Free, large reach, official content	Weak UX for daily teaching tasks; no AI assistance tailored to a teacher's actual question
Content/smart-class providers	Extramarks-type	Curriculum video content	Static content, expensive, not generative or personalized

12.1 Where SarasTech can credibly win

- 1. The teacher's real workflow, in Indian formats and languages.** Lesson plans in the prescribed format with a blackboard summary, two-column presentation, multi-grade classrooms, Hinglish. This is already a real, hard-to-copy head start.

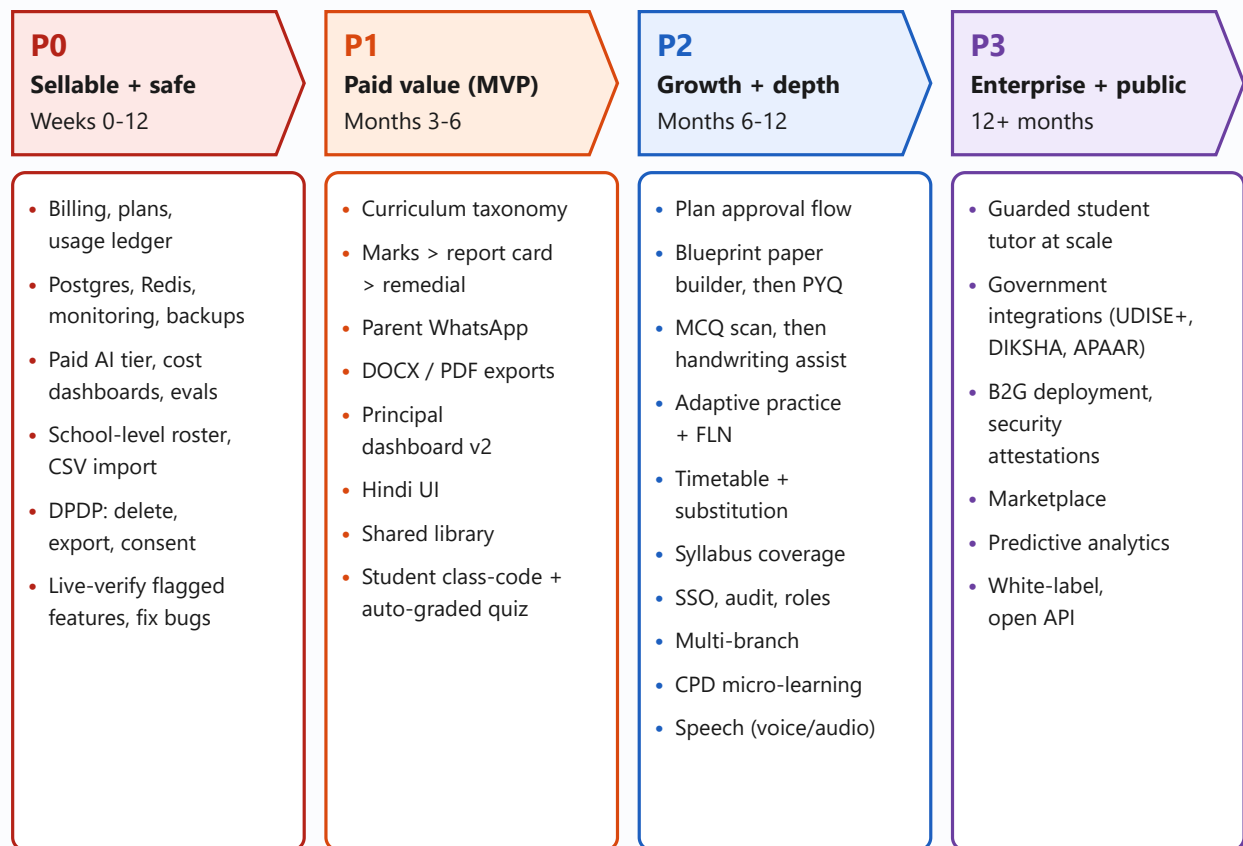
2. **AI + school operations in one product.** Teacher attendance (geofenced, reviewed) + student attendance + fees + lesson plans + AI generation. ERP vendors lack the AI; AI vendors lack the operations.
3. **Trust engineering as a feature.** Answer-key separation that fails closed, deterministic question numbering, schema-validated output, an eval harness, prompt-injection separation. Market it: *"AI you can print without re-checking every line"* (only after P0-7 makes it true).
4. **The assessment → marks → remediation loop** that generic tools cannot do because they lack the roster and results data.
5. **Vernacular-first, low-bandwidth, WhatsApp-native** experience for teachers and parents.

12.2 Defensibility (be honest)

Model access is a commodity. The moat is **(a)** curriculum taxonomy and reviewed content, **(b)** school-level workflow data and integrations, **(c)** eval-verified quality in Indian languages, **(d)** distribution/relationships, and **(e)** switching cost from being the school's system of record. Features that depend only on prompting a model should be assumed copyable within months.

13. Roadmap

Figure 4 - P0 to P3 roadmap



P0 goal: run 3-10 paid pilot schools. P1 exit: renewing schools, measurable hours saved, margin floor.

Why this order (dependencies)



Student identity stays minimal until consent and roster foundations are solid; WhatsApp can ship before the student portal.

Phased roadmap with the dependency logic behind the ordering.

13.1 Phase 0 — "Sellable and safe" (weeks 0–12)

Goal: run 3–10 paid pilot schools without embarrassment.

Track	Items
Revenue	P0-1 billing/entitlements/usage ledger (Razorpay, GST invoices); plan gating of expensive features
Platform	P0-2 Postgres (when you initiate), Redis limits/budgets, staging, monitoring, backups, migration-as-release-step
AI	P0-3 paid tier + cost dashboards; P0-7 answer-key self-check and eval suites; measure real cost per feature

Track	Items
Data model	P0-5 school-level roster, academic year, teacher assignments, CSV import, migration of existing teacher-owned classes
Compliance	P0-4 deletion/export, consent capture, DPA template, PII-not-sent-to-model test
Launch hygiene	P0-6 live-verify flagged features, fix offline-restore bug, refresh README, fix truncated admin analytics
Discovery (non-code)	15–25 interviews across principals, teachers, parents; confirm ICP and price points

13.2 Phase 1 — MVP for paid value (months 3–6)

P1-1 curriculum taxonomy (CBSE + 1–2 boards) · P1-2 marks → report card → remedial · P1-3 parent WhatsApp (attendance/fees/results) · P1-6 DOCX/PDF exports · P1-7 principal dashboard v2 · P1-8 Hindi UI · P1-5 shared library (view/copy) · P1-4 student class-code + assigned auto-graded quiz (MVP scope). **Exit criteria:** ≥ N paying schools renewing, measurable teacher hours saved (self-reported and log-derived), weekly-active-teacher rate ≥ agreed target, gross margin per school ≥ agreed floor after AI + messaging costs.

13.3 Phase 2 — Growth and depth (months 6–12)

P1-5 plan approval workflow · P2-1 blueprint paper builder (then PYQ) · P2-2 MCQ/OMR scan then handwriting assist · P2-3 adaptive practice + FLN baseline/endline · P2-4 timetable/substitution/syllabus coverage · P2-6 SSO/audit/custom roles · P2-7 multi-branch · P2-8 CPD · P2-9 speech · more UI languages and boards.

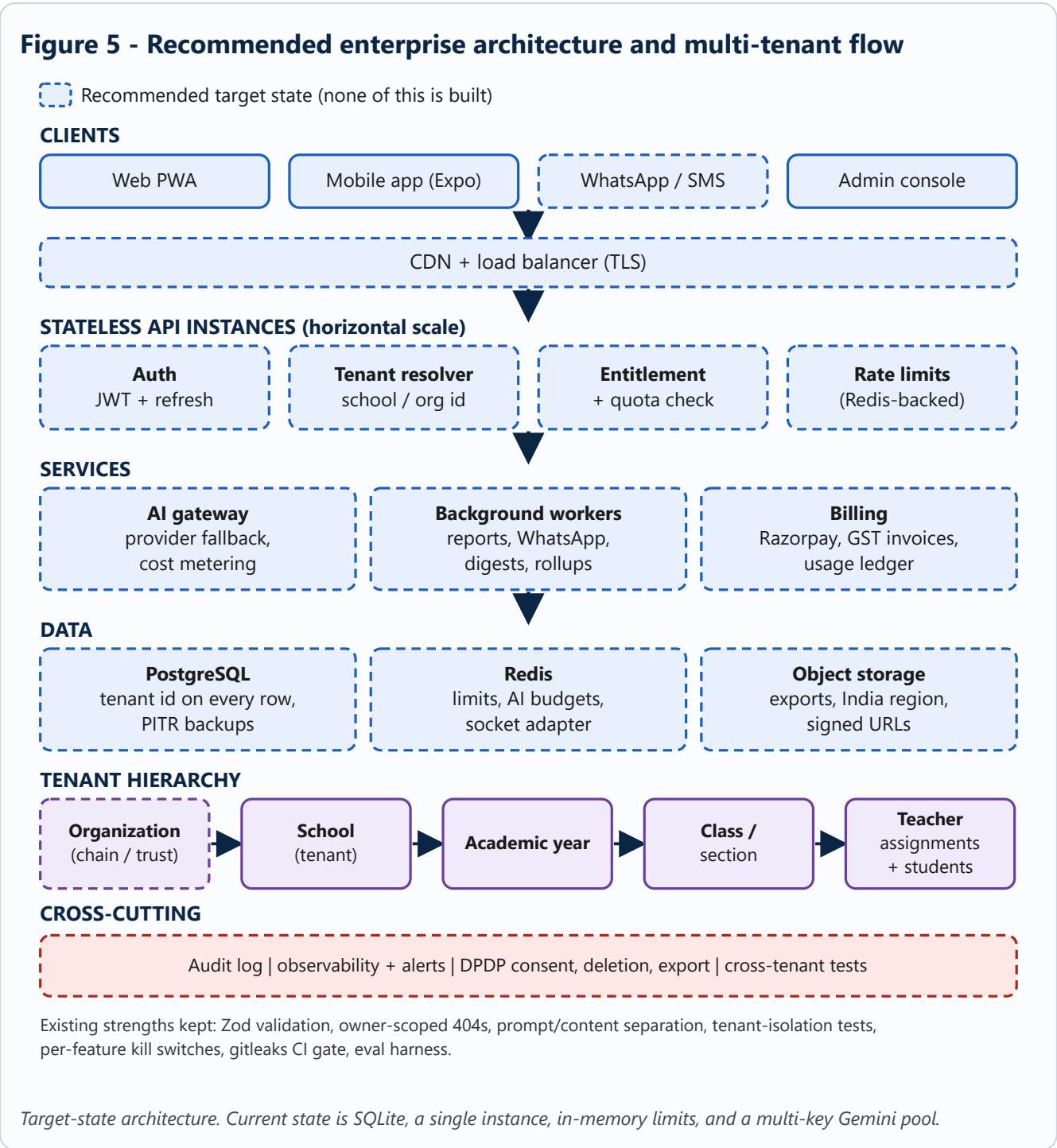
13.4 Phase 3 — Enterprise and public sector (12+ months)

Guarded student tutor (P2-5 at scale), government integrations, B2G deployment options and security attestations (VAPT, ISO 27001/SOC 2-style program as customers demand), marketplace, predictive analytics, white-label, open API.

13.5 Sequencing logic (why this order)

- P0-5 (school data model) must precede nearly everything in P1: every additional feature written against `teacherId`-scoped tables increases migration cost.
- P1-1 (curriculum taxonomy) is a dependency for gradebook-by-topic, PYQ, remediation, and syllabus tracking; start data sourcing during Phase 0.
- Student identity (P1-4) should be minimal until consent and roster foundations are solid.
- WhatsApp (P1-3) is a high-visibility win that needs P0-4/P0-5 but not the student portal, so it can ship earlier than student features.

14. Architecture blockers as we scale (detail)



Area	Current	Blocker at scale	Direction
Database	SQLite, one file, migrate-on-start	One instance only; no PITR; concurrent-write ceiling; multi-instance deploy race	Managed Postgres; separate migrate job; connection pooling; read replica for reporting later
Rate limits / AI budgets	express-rate-limit memory store; budget.js in-process Map	Multiplies with instance count; resets on deploy; cannot back billing	Redis-backed limiter and metering; ledger in DB
Realtime	Socket.IO single process	Notifications lost across instances	Redis adapter or move to push + polling fallback

Area	Current	Blocker at scale	Direction
Tenancy	<code>School</code> is tenant; classroom tables scoped by <code>teacherId</code>	No institutional data; no org hierarchy	School-owned roster; <code>Organization</code> above <code>School</code>
Roles/permissions	4 roles, duplicated in server/web/mobile (CHANGE-11)	Cannot add student/parent/HOD/accountant without triple edits	Shared TypeScript package for vocab; permission-based checks (role → permission set); per-tenant custom roles later
Feature gating	Env flags + school-code allowlists + <code>SystemSetting</code>	Cannot express plans	Entitlement resolver (P0-1)
AI	One vendor, multi-key pool; prompts in code	Vendor outage/price change; no cost accounting	Provider abstraction with fallback; per-feature cost telemetry; prompt versioning tied to evals
Analytics	JS reduction over ≤5,000 rows	Silently wrong for big tenants	SQL aggregation / summary tables; scheduled rollups
Content model	<code>Resource.content</code> Markdown + <code>structured JSON</code>	No sharing, versions, or comments	Add <code>ResourceShare</code> , <code>ResourceVersion</code> , review states
Files	No persistent storage by design	Report cards/exports/uploads need it	Object storage (S3-compatible, India region) with signed URLs and retention
Codebase	Web and mobile duplicate types/vocab; root <code>src/ / test/</code> empty; large files (<code>resources.js</code> ~1.7k lines, <code>index.js</code> ~1k lines with inline <code>/coach</code>)	Slows feature velocity and increases regression risk	Extract shared package; split routes/services; keep the strong test discipline
Client i18n	None	Blocks vernacular UI	i18n framework + shared key catalog
Ops	Hosting billing lapse blocked verification; no staging described	Single-point-of-failure operations	Separate prod/staging, alerts, on-call rota, status page
Testing	Strong server suite; e2e limited; no load tests	Unknown behavior under Monday-morning load	Load tests; contract tests for mobile/web/server; more e2e of critical paths (check-in, attendance, generation, billing)

Note on existing strengths to preserve: Zod-strict validation, owner-scoped 404s, prompt/content separation, deterministic document rendering, tenant-isolation tests, kill switches per feature, gitleaks CI gate, and the eval harness. These are the reason the platform is a good base; scaling work should extend them, not replace them.

15. Security, privacy, scalability, reliability, billing, and multi-tenant requirements

15.1 Security

- **Already good:** server-side AI key, JWT + rotating refresh, logout, RBAC on admin routes, 404 non-disclosure, Zod validation, helmet/CORS allowlist, secret scanning.
- **Open items:** git history still contains historical AI keys per `docs/git-history-secret-purge.md` (rotate regardless; purge runbook not executed); the seeded `demo1234` accounts must never exist in production; no MFA for admins; no admin-action audit log beyond selected events; no documented VAPT; tokens stored in `localStorage` (XSS blast radius: consider httpOnly cookie strategy for web if feasible, weigh against the existing bearer design); dependency/SAST scanning in CI beyond gitleaks; file-upload hardening review as storage is introduced; rate limits per tenant and per user for AI features (moves to Redis).
- **For enterprise sales:** third-party penetration test report, security whitepaper, vulnerability disclosure policy, incident response runbook, backup/restore drills.

15.2 Privacy (children's data)

- Treat the school as the data fiduciary and SarasTech as processor where appropriate; DPA; consent records; parental consent flow for any student-facing feature; data minimization (do not collect DOB/phone until needed); never send student PII to the model; retention and deletion schedules; India data residency options; clear privacy notices in local languages; no ads or third-party trackers for student data. **Obtain legal advice on DPDP obligations.**

15.3 Scalability

- Stateless API instances behind a load balancer; Postgres + Redis; background job queue (report generation, WhatsApp sends, digests, rollups, PYQ extraction); CDN for the static client; per-tenant quotas; bulk operations chunked; pagination everywhere (admin lists already paginated: extend to all list endpoints); caching for read-heavy public/SEO pages.

15.4 Reliability

- Define SLOs (e.g. attendance/check-in success rate and p95 latency); graceful AI degradation (fallback provider, queue-and-notify for slow generations, cached last-good outputs); offline-first for attendance/marks/student quizzes with conflict resolution; idempotency keys on writes that may be retried from mobile; status page; tested backup restore.

15.5 Billing and subscription

- Razorpay (or comparable) with UPI autopay, GST invoices, proration, trials, grace periods, failed-payment dunning, seat management, plan changes, refunds, credit notes, purchase-order/invoice-based billing for institutions; entitlement checks server-side only (never trust the client); usage ledger with reconciliation; admin billing screens; revenue metrics (MRR, churn, ARPA, cost per active teacher).

15.6 Multi-tenant requirements

- Tenant ID on every tenant-owned row and in every query path; automated cross-tenant leakage tests for **every** new endpoint (existing pattern); per-tenant configuration (academic calendar, grading scheme, letterhead, timezone/locale, fee heads); per-tenant feature entitlements; tenant-scoped exports and deletion;

noisy-neighbor protection (per-tenant rate limits); support tooling with audited, time-boxed impersonation; organization hierarchy for chains.

16. Risks and open questions

Risk / question	Why it matters	Mitigation / next step
ICP unclear (government vs. private)	Determines pricing, sales cycle, features	Customer discovery; commit to ICP-1 for 2 quarters
AI answer errors in printed material	Churn and reputational damage	P0-7 evals, self-check, easy error reporting
AI/WhatsApp unit costs unmeasured	Could make plans unprofitable	Measure in Phase 0; meter and cap
Children's data compliance	Legal and trust	P0-4 and legal counsel before any student-facing launch
Content licensing (textbooks, PYQ)	Legal exposure and content-ops cost	Legal review; start with open/permitted content; partnerships
Vendor dependence (single LLM provider)	Outage/price/terms risk	Fallback provider behind <code>gemini.js</code> seam
Schema migration to school-owned data	High-risk, touches most feature tables	Do it first, with backfill scripts and rollback plan
Team capacity	Roadmap is broad	Keep P0/P1 strict; say no to P2/P3 until pilots renew
Hosting/billing lapses (already occurred)	Operational credibility	Move to managed hosting with prepaid/auto-pay, alerts, and a second person with access
Feature sprawl behind flags	Many features never reach users	Time-box flag decisions in P0-6; delete or defer what pilots ignore

16.1 Metrics to track from day one

Weekly active teachers per school; generations per teacher per week; % of generated items actually saved/printed/shared; time-to-first-value for a new school; attendance-marking adherence; report-card completion rate; principal dashboard opens per week; parent message delivery/read rate; answer-key error reports per 1,000 generations; gross margin per school; logo and seat retention; NPS from teachers and principals; support tickets per school.

16.2 Questions for the founders/product owner

1. Which segment do you commit to first (private low-fee schools, coaching/tuition, government via NGO/CSR)?
2. Is there an existing pilot school set, and what do they say they would pay for?
3. Are you willing to sanction the Postgres migration and Redis introduction now (currently deferred by standing instruction)?
4. Is a legal entity, GST registration, and payment-gateway account in place?
5. What data-residency commitment (if any) will you offer?
6. Who owns content operations (taxonomy, PYQ review, translation review)?

17. Appendix: evidence map (where the claims above come from)

Claim	Source
Models and scoping (teacherId only for classroom data; schoolId reserved)	server/prisma/schema.prisma (SchoolClass , Student , AttendanceRecord , FeeRecord)
Roles are four hard-coded strings and duplicated	server/src/lib/roles.js , client/src/config.ts comments
Coarse grade/subject vocabularies	client/src/config.ts (GRADES , SUBJECTS)
Feature flags and default-off rollout	server/src/lib/flags.js ; README rollout section
Per-process AI budget	server/src/assistant/budget.js header comment
Truncated analytics (take: 5000)	server/src/routes/admin.js /analytics
Multi-key Gemini rotation	docs/multiple_api_key.md , server/src/lib/geminiKeyPool.js
No Redis for Socket.IO	server/src/lib/socketServer.js header
PYQ is plan-only	docs/pyq-implementation-plan.md status banner; no PYQ code in server/src
Classroom Mode: 4 calls per question, live verification outstanding	docs/classroom-mode.md §1
Indian lesson-plan format (objectives, TLM, presentation, blackboard summary)	server/src/lib/lessonPlanSchema.js
Teacher attendance features	server/prisma/schema.prisma (SchoolAttendanceConfig , TeacherAttendance*), server/src/routes/teacherAttendance.js
No billing/payment code	repository-wide grep of server/src , client/src , mobile/src
No i18n framework	grep of client/src , mobile/src , client/package.json
Offline restore bug; hosting billing lapse	README.md Mobile App section
Prior engineering audit (security/scale findings)	docs/enterprise-engineering-audit.md
Existing exports (Excel for attendance/fees, CSV)	server/package.json (exceljs), server/src/lib/*ReportExcel.js , csv.js

End of document. Update this file as priorities are decided, and record dated decisions in a change log at the top rather than rewriting history.